

EXPERIMENT - 4

EXPERIMENT - 4

Aim

To capture, identify, and analyze various network protocols using Wireshark and to study at least five important protocols (TCP, UDP, ARP, ICMP, HTTP) in detail using display filters and flow graphs.

Requirements

- System with Internet connectivity
- Wireshark installed
- Web browser
- Command Prompt / Terminal

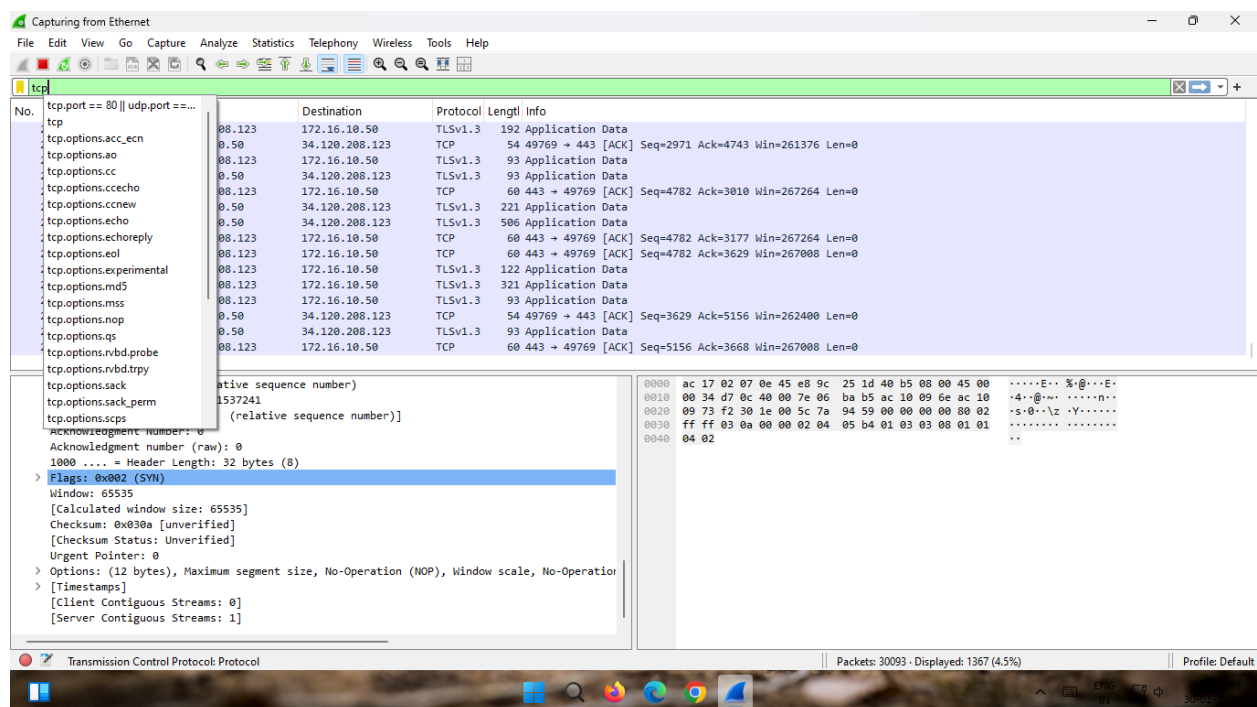
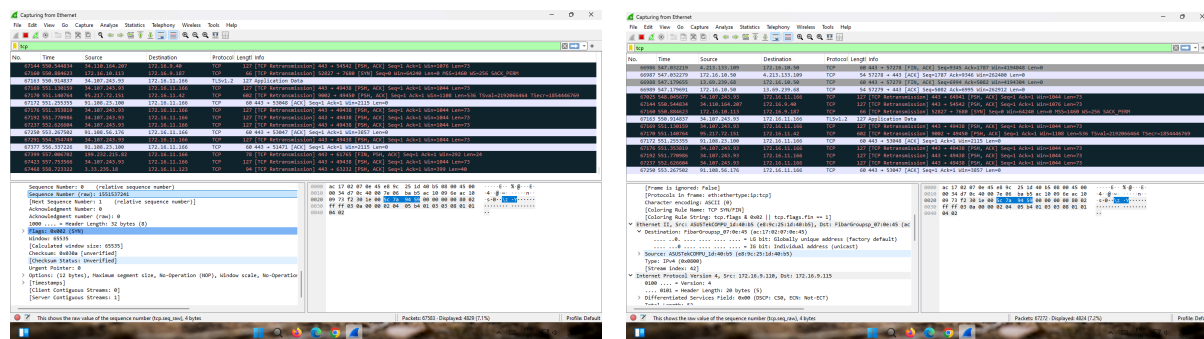
Procedure

1. Install and open Wireshark on the system.
2. Select the active network interface (Wi-Fi/Ethernet).
3. Click on **Start Capture** to begin packet capturing.
4. Generate network traffic by browsing a website or using commands like `ping` and `nslookup`.
5. Stop the packet capture after sufficient packets are recorded.
6. Apply display filters such as `tcp`, `udp`, `arp`, `icmp`, and `http` to analyze specific protocols.
7. Select individual packets to examine header details in the packet details pane.
8. Use **Follow** → **TCP Stream** to observe complete TCP communication.

9. Open **Statistics** → **Flow Graph** to visualize packet flow between source and destination.

10. Analyze the captured packets and record observations for each protocol.

OUTPUT



Observation

- TCP showed three steps before sending data (SYN, SYN-ACK, ACK).
- UDP sent data directly without any connection setup.
- ARP asked for a MAC address and received a reply.
- ICMP showed request and reply packets during ping.
- HTTP showed website request and server response.

RESULT

The network protocols TCP, UDP, ARP, ICMP, and HTTP were successfully captured and analyzed using Wireshark. The packet details, filters, and flow graph helped in understanding how data is transmitted between source and destination. The working and behavior of each protocol were studied clearly and verified.

EXPERIMENT - 5

EXPERIMENT - 5

Aim

To design a Hamming Code sender utility and to generate an error-detecting bitstream using even parity.

Procedure

1. Input the binary data string.
2. Calculate required check bits using $2^r \geq m + r + 1$ where m is the number of data bits and r is the number of check bits.
3. Place check bits at positions 1, 2, 4, 8, etc.
4. Insert data bits in remaining positions.
5. Assign each data bit to check bit groups based on binary position.
6. Compute check bits to maintain even parity.
7. Combine all bits to form final encoded bitstream.

Program

```
def calculate_parity_bits(m):  
    p = 0  
    while (2 ** p) < m + p + 1:  
        p += 1  
    return p  
  
def position_parity_bits(data, p):  
    j = 0  
    k = 1  
    m = len(data)
```

```
result = ''
```

```
for i in range(1, m + p + 1):
```

```
    if i == 2 ** j:
```

```
        result += '0'
```

```
        j += 1
```

```
    else:
```

```
        result += data[-k]
```

```
        k += 1
```

```
return result[::-1]
```

```
def calculate_parity_values(code, p):
```

```
    n = len(code)
```

```
    for i in range(p):
```

```
        val = 0
```

```
        for j in range(1, n + 1):
```

```
            if j & (2 ** i) == (2 ** i):
```

```
                val = val ^ int(code[-j])
```

```
        code = code[:n-(2**i)] + str(val) + code[n-(2**i)+1:]
```

```
    return code
```

```
def detect_correct_error(code, p):
```

```
    n = len(code)
```



```

error_pos = 0

for i in range(p):

    val = 0

    for j in range(1, n + 1):

        if j & (2 ** i) == (2 ** i):

            val = val ^ int(code[-j])

    error_pos += val * (2 ** i)

if error_pos:

    print(f"Error found at position: {error_pos}")

    corrected_code = code[:n - error_pos] + str(1 - int(code[n -
error_pos])) + code[n - error_pos + 1:]

    return corrected_code

else:

    print("No error detected.")

    return code

def main():

    original_data = input("Enter the data bits: ")

    m = len(original_data)

    p = calculate_parity_bits(m)

    positioned_data = position_parity_bits(original_data, p)

    hamming_code = calculate_parity_values(positioned_data, p)

```

```
print(f"Hamming Code: {hamming_code}")

received_code = input("Enter the received Hamming Code: ")

corrected_code = detect_correct_error(received_code, p)

print(f"Corrected Hamming Code: {corrected_code}")


if __name__ == "__main__":

    main()
```

Output

```
Enter the data bits: 1001010
Hamming Code: 10011010001
Enter the received Hamming Code: 10011010001
No error detected.
Corrected Hamming Code: 10011010001
```

Observation

1. The data length increases from 4 bits to 7 bits due to added check bits.
2. Check bits are placed at positions 1, 2, and 4, forming a structured pattern.
3. Each check bit covers multiple data bits, creating overlapping groups.
4. Even parity maintains a balanced number of 1s in each group.
5. The final encoded output is 0110011, ensuring error detection capability.

Result

Hamming code successfully generated with even parity.

Experiment-6

Experiment-6

Aim

To simulate the **Sliding Window Protocol** for data transmission.

Procedure

- Start the program.
- Input the **window size (w)**.
- Enter the **number of frames (f)** to be transmitted.
- Input the sequence of frames.
- The sender transmits frames in groups equal to the window size.
- After sending each window, the sender waits for acknowledgement.
- Continue until all frames are transmitted.

Output

[illegible]

Observation

- Frames are sent in groups based on window size.
- Acknowledgement is received after each group of frames.
- Sender waits only after sending a full window.
- Remaining frames are sent if frames are not multiple of window size.
- Transmission efficiency is improved compared to one-by-one sending.

Result

Sliding Window Protocol is successfully simulated with efficient frame transmission using grouped acknowledgements.

Experiment-7

Experiment-7

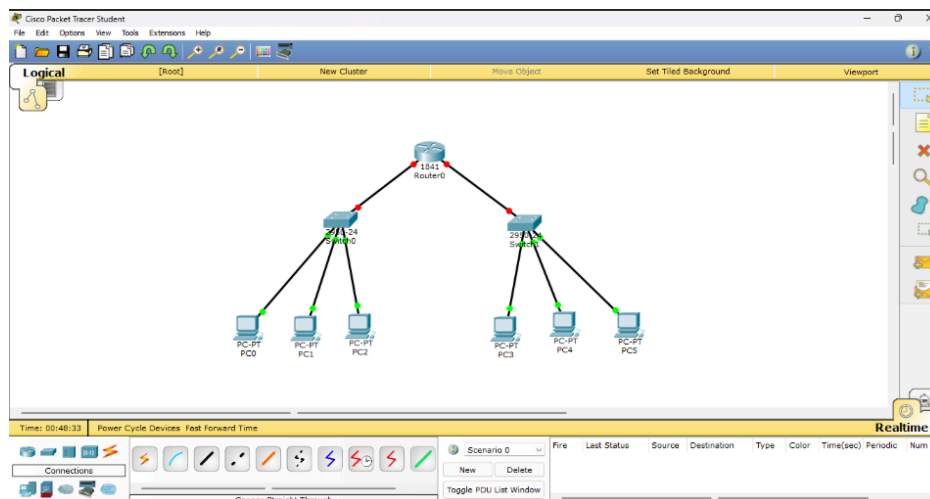
Aim

To design and configure **2 subnets** from the given network 192.168.10.0/25.

Procedure

1. Given network: **192.168.10.0/25** (128 IP addresses).
2. Divide the network into **2 equal subnets**.
3. Subnet mask becomes **/26 (255.255.255.192)**.
4. Subnet 1:
 - o Network ID: **192.168.10.0/26**
 - o Usable IPs: 192.168.10.1 – 192.168.10.62
5. Subnet 2:
 - o Network ID: **192.168.10.64/26**
 - o Usable IPs: 192.168.10.65 – 192.168.10.126
6. Assign at least 3 IP addresses to computers in each subnet.
7. Configure IP address and subnet mask in each system.
8. Use a router or switch to connect subnets.
9. Send PDU (ping) packets to test connectivity.

Output



Observation

1. The given network is successfully divided into two equal subnets.
2. Each subnet supports more than 3 computers.
3. Devices within the same subnet communicate directly.
4. Communication between subnets requires routing.
5. PDU packets confirm successful data transmission.

Result

Thus Subnets are successfully created and configured.

Experiment-8

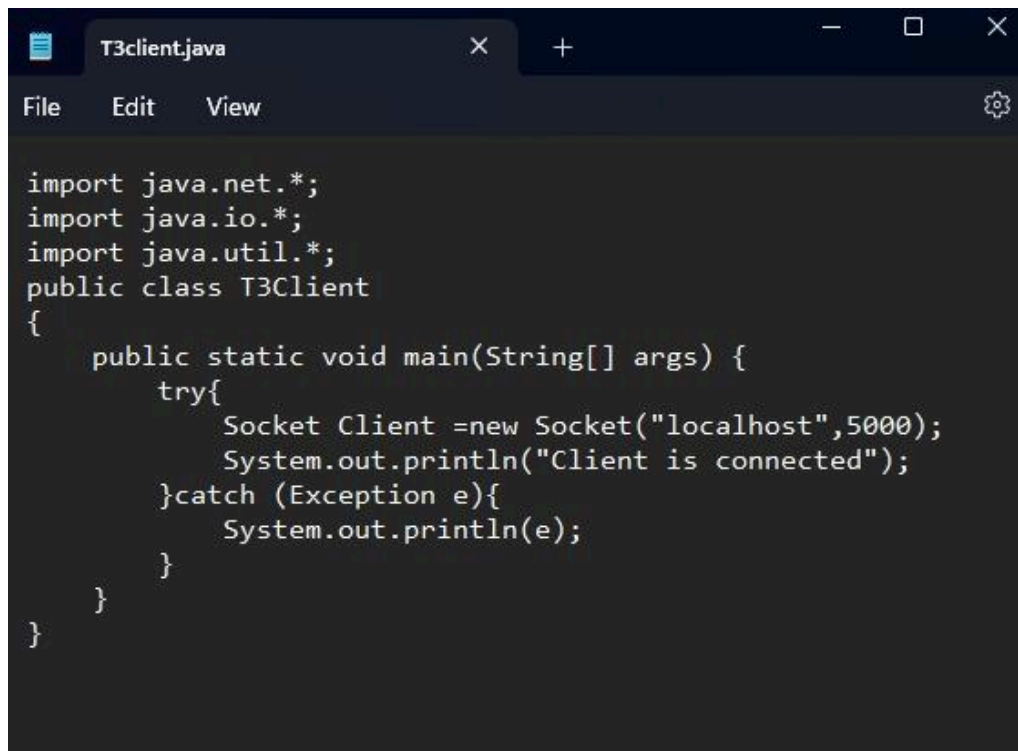
Experiment-8

Aim

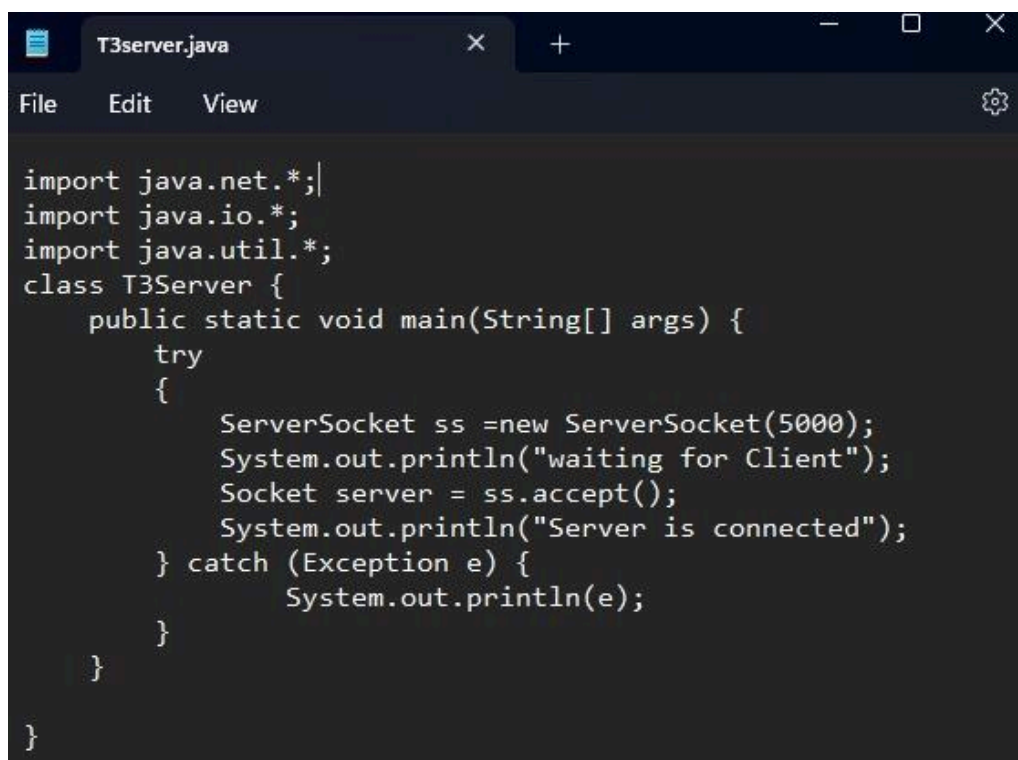
To implement an Echo Client-Server communication using TCP and UDP sockets in Java, where the client sends a message and the server echoes it back.

Procedure

- Start the program execution.
- Create a server program:
 - For TCP, initialize a `ServerSocket` with a port number.
 - For UDP, initialize a `DatagramSocket` with a port number.
- Wait for client communication:
 - In TCP, accept the client connection using `accept()`.
 - In UDP, wait to receive a packet using `receive()`.
- Create a client program:
 - For TCP, establish connection using `Socket` with server IP and port.
 - For UDP, create a `DatagramSocket` and prepare a packet with server address.
- Send message from client to server.
- Receive the message at the server side.
- Echo the same message back to the client:
 - In TCP, send via output stream.
 - In UDP, send using `DatagramPacket`.
- Receive echoed message at the client side and display it.
- Repeat communication if required until termination.
- Close all sockets and end the program.



```
import java.net.*;
import java.io.*;
import java.util.*;
public class T3Client
{
    public static void main(String[] args) {
        try{
            Socket Client =new Socket("localhost",5000);
            System.out.println("Client is connected");
        }catch (Exception e){
            System.out.println(e);
        }
    }
}
```



```
import java.net.*;
import java.io.*;
import java.util.*;
class T3Server {
    public static void main(String[] args) {
        try
        {
            ServerSocket ss =new ServerSocket(5000);
            System.out.println("waiting for Client");
            Socket server = ss.accept();
            System.out.println("Server is connected");
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

```
Command Prompt
Microsoft Windows [Version 10.0.22631.5039]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Tcs>cd desktop

C:\Users\Tcs\Desktop>javac T3Client.java

C:\Users\Tcs\Desktop>java T3Client
Client is connected

C:\Users\Tcs\Desktop>|
```

Observation

- In **TCP**, connection is established before communication.
- Messages are delivered reliably and in order.
- In **UDP**, no connection setup is required.
- Data may be faster but not guaranteed to reach.
- Both programs successfully echoed client messages.

Result

The Echo Client-Server programs using both **TCP and UDP sockets** were successfully implemented and executed. The server correctly received messages from the client and sent back the same message as output.

Experiment-9

Experiment-9

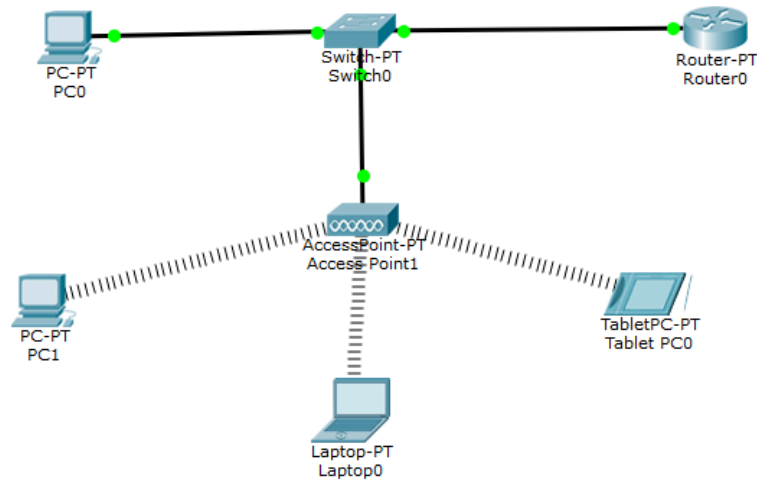
Aim

To study and configure a wireless router and establish a wireless network connection between devices.

Procedure

- Switch on the wireless router and connect it to the power supply.
- Connect the router to the internet source (modem or ISP cable).
- Use a computer/mobile device to search for available Wi-Fi networks.
- Connect to the router's default SSID (network name).
- Open a web browser and enter the router's IP address (e.g., 192.168.1.1).
- Log in using default username and password.
- Configure basic settings:
 - Change SSID (network name).
 - Set a secure password (WPA2/WPA3).
- Save settings and restart the router if required.
- Reconnect devices using the new SSID and password.
- Test the connection by accessing websites or transferring data.

Output



Observation

- The wireless router broadcasts an SSID that devices can detect.
- Devices successfully connected using the configured password.
- Data transmission occurred without physical cables.
- Network speed and signal strength varied based on distance and obstacles.

Result

The wireless router was successfully configured, and a stable wireless network connection was established between devices, enabling internet access and communication.